Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import GRU, LSTM, Dense, Dropout, Conv1D, MaxPo
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.optimizers import Adam
from sklearn.preprocessing import RobustScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_scor
import warnings
import re

# Configuration
warnings.filterwarnings('ignore')
np.random.seed(42)
tf.random.set_seed(42)

# Set plot style
sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (12, 6)

print("="*80)
print("TRAFFIC CONGESTION PREDICTION - COMPLETE PIPELINE")
print("="*80)
print("\n✓ Libraries imported successfully")
print(f"✓ TensorFlow version: {tf.__version__}")
print(f"✓ NumPy version: {np.__version__}")
print(f"✓ Pandas version: {pd.__version__}")
```

```
================================================================================
TRAFFIC CONGESTION PREDICTION - COMPLETE PIPELINE
================================================================================

✓ Libraries imported successfully
✓ TensorFlow version: 2.19.0
✓ NumPy version: 2.0.2
✓ Pandas version: 2.2.2
```

Load Dataset

```
print("\n" + "="*80)
print("STEP 1: DATA LOADING")
print("="*80)

df = pd.read_csv("traffic_weather_full2020.csv")
```

```
print(f"\n✓ Dataset loaded successfully")
print(f"  Shape: {df.shape}")
print(f"  Time range: {df['5 Minutes'].min()} to {df['5 Minutes'].max()}")
print(f"\nFirst 3 rows:")
print(df.head(3))
```

```
================================================================================
STEP 1: DATA LOADING
================================================================================

✓ Dataset loaded successfully
  Shape: (34823, 17)
  Time range: 2/1/2020 0:00 to 5/9/2020 9:55

First 3 rows:
       5 Minutes  Flow  (mph)  week_day  holidays           Hour       Day  \
0  2/1/2020 0:00   338   66.0         6         1  2/1/2020 0:00  2/1/2020
1  2/1/2020 0:05   360   65.5         6         1  2/1/2020 0:05  2/1/2020
2  2/1/2020 0:10   347   65.2         6         1  2/1/2020 0:10  2/1/2020

    Time Temperature Dew Point  Humidity Wind Wind Speed Wind Gust  Pressure
0  12:00        55 F      29 F      37.0  VAR          3         0     29.39
1  12:00        55 F      29 F      37.0  VAR          3         0     29.39
2  12:00        55 F      29 F      37.0  VAR          3         0     29.39

   Precip. Condition
0      0.0      Fair
1      0.0      Fair
2      0.0      Fair
```

Data Cleaning - Column Renaming and Timestamp

```
print("\n" + "="*80)
print("STEP 2: DATA CLEANING")
print("="*80)

# Rename problematic column
df = df.rename(columns={'(mph)': 'speed_mph'})
print("\n✓ Renamed '(mph)' to 'speed_mph'")

# Convert timestamp
df['timestamp'] = pd.to_datetime(df['5 Minutes'])
df = df.sort_values('timestamp').reset_index(drop=True)
print("✓ Converted timestamp and sorted chronologically")
```

```
================================================================================
STEP 2: DATA CLEANING
================================================================================

✓ Renamed '(mph)' to 'speed_mph'
✓ Converted timestamp and sorted chronologically
```

Data Cleaning - Numeric Conversion Function

```
def clean_numeric(x):
    """
    Remove non-numeric characters from strings
    Keeps digits, decimal point, and minus sign
    """
    if isinstance(x, str):
        x = re.sub(r'[^0-9.\-]', '', x)
        return float(x) if x != '' else np.nan
    return float(x) if pd.notna(x) else np.nan

print("\n✓ Cleaning function defined")
```

```
✓ Cleaning function defined
```

Data Cleaning - Apply Numeric Conversion

```
# Clean temperature fields
df['Temperature'] = df['Temperature'].apply(clean_numeric)
df['Dew Point'] = df['Dew Point'].apply(clean_numeric)

print("\n✓ Cleaned temperature fields")
print(f"  Temperature range: {df['Temperature'].min():.1f} to {df['Temperatu

# Clean other numeric fields
numeric_fields = ['Wind Speed', 'Wind Gust', 'Pressure', 'Precip.', 'Humidit
for field in numeric_fields:
    df[field] = df[field].apply(clean_numeric)

print(f"✓ Cleaned {len(numeric_fields)} additional numeric fields")
```

```
✓ Cleaned temperature fields
  Temperature range: 35.0 to 98.0
✓ Cleaned 5 additional numeric fields
```

Data Cleaning - Handle Missing Values

```
print("\nMissing values before cleaning:")
missing_before = df.isnull().sum()
print(missing_before[missing_before > 0])

# Fill numeric columns with median
numeric_cols = df.select_dtypes(include=[np.number]).columns
for col in numeric_cols:
    if df[col].isnull().sum() > 0:
        df[col].fillna(df[col].median(), inplace=True)
```

```
# Fill categorical columns with mode
categorical_cols = ['Wind', 'Condition']
for col in categorical_cols:
    if col in df.columns and df[col].isnull().sum() > 0:
        df[col].fillna(df[col].mode()[0], inplace=True)

print(f"\n✓ Missing values after cleaning: {df.isnull().sum().sum()}")
```

```
Missing values before cleaning:
Wind         168
Condition     24
dtype: int64

✓ Missing values after cleaning: 0
```

Data Cleaning - Drop Redundant Columns

```
# Drop columns that are duplicates or not needed
cols_to_drop = ['5 Minutes', 'Hour', 'Day', 'Time']
df = df.drop(columns=[col for col in cols_to_drop if col in df.columns])

print(f"\n✓ Dropped redundant columns")
print(f"  Final shape after cleaning: {df.shape}")
print(f"  Remaining columns: {len(df.columns)}")
```

```
✓ Dropped redundant columns
  Final shape after cleaning: (34823, 14)
  Remaining columns: 14
```

Feature Engineering - One-Hot Encoding

```
print("\n" + "="*80)
print("STEP 3: FEATURE ENGINEERING")
print("="*80)

df_features = df.copy()

# One-hot encode Wind direction
if 'Wind' in df_features.columns:
    wind_dummies = pd.get_dummies(df_features['Wind'], prefix='wind', drop_f
    df_features = pd.concat([df_features, wind_dummies], axis=1)
    df_features = df_features.drop(columns=['Wind'])
    print(f"\n✓ Wind directions encoded: {len(wind_dummies.columns)} dummy v

# One-hot encode Weather condition
if 'Condition' in df_features.columns:
    condition_dummies = pd.get_dummies(df_features['Condition'], prefix='con
    df_features = pd.concat([df_features, condition_dummies], axis=1)
    df_features = df_features.drop(columns=['Condition'])
    print(f"✓ Weather conditions encoded: {len(condition_dummies.columns)} c
```

```
================================================================================
STEP 3: FEATURE ENGINEERING
================================================================================

✓ Wind directions encoded: 17 dummy variables
✓ Weather conditions encoded: 14 dummy variables
```

Feature Engineering - Time-based Features

```
print("\n✓ Creating time-based features...")

# Extract time components
df_features['hour'] = df_features['timestamp'].dt.hour
df_features['day_of_week'] = df_features['timestamp'].dt.dayofweek

# Cyclical encoding
df_features['hour_sin'] = np.sin(2 * np.pi * df_features['hour'] / 24)
df_features['hour_cos'] = np.cos(2 * np.pi * df_features['hour'] / 24)

print("  Created: hour, day_of_week, hour_sin, hour_cos")
```

```
✓ Creating time-based features...
  Created: hour, day_of_week, hour_sin, hour_cos
```

Feature Engineering - Lag Features

```
print("\n✓ Creating lag features...")

# Skip 5-min lag to avoid data leakage for 1-hour prediction
lags = [3, 6, 12, 24]  # 15min, 30min, 1hr, 2hr
```

```
for lag in lags:
    df_features[f'flow_lag_{lag}'] = df_features['Flow'].shift(lag)
    minutes = lag * 5
    print(f"  Created flow_lag_{lag} ({minutes} minutes ago)")
```

```
✓ Creating lag features...
  Created flow_lag_3 (15 minutes ago)
  Created flow_lag_6 (30 minutes ago)
  Created flow_lag_12 (60 minutes ago)
  Created flow_lag_24 (120 minutes ago)
```

Feature Engineering - Rolling Features

```
print("\n✓ Creating rolling features (past-only)...")

# 30-minute rolling window
df_features['flow_roll_mean_6'] = df_features['Flow'].shift(1).rolling(6).me
df_features['flow_roll_std_6'] = df_features['Flow'].shift(1).rolling(6).std

print("  Created: rolling mean and std (30-minute window)")
```

```
✓ Creating rolling features (past-only)...
  Created: rolling mean and std (30-minute window)
```

Feature Engineering - Finalize

```
print(f"\nShape before dropping NaN: {df_features.shape}")

df_features = df_features.dropna()

print(f"Shape after dropping NaN: {df_features.shape}")
print(f"✓ Total features created: {len(df_features.columns)}")
print("✓ Feature engineering complete")
```

```
Shape before dropping NaN: (34823, 53)
Shape after dropping NaN: (34799, 53)
✓ Total features created: 53
✓ Feature engineering complete
```

Sequence Creation

```
print("\n" + "="*80)
print("STEP 4: SEQUENCE CREATION")
print("="*80)

# Configuration
INPUT_WINDOW = 72       # 6 hours of history (72 * 5 min)
```

```
OUTPUT_HORIZON = 12     # 1 hour ahead (12 * 5 min = 60 minutes)

print(f"\nSequence Configuration:")
print(f"  Input window: {INPUT_WINDOW} timesteps = {INPUT_WINDOW * 5 / 60:.1
print(f"  Output horizon: {OUTPUT_HORIZON} timesteps = {OUTPUT_HORIZON * 5}

# Select features
feature_cols = [col for col in df_features.columns
                if col not in ['timestamp', 'Flow', 'speed_mph']]
target_col = 'Flow'

print(f"\n✓ Features: {len(feature_cols)}")
print(f"✓ Target: {target_col}")

# Build sequences
X, y = [], []
for i in range(len(df_features) - INPUT_WINDOW - OUTPUT_HORIZON):
    X.append(df_features[feature_cols].iloc[i:i+INPUT_WINDOW].values)
    y.append(df_features['Flow'].iloc[i+INPUT_WINDOW:i+INPUT_WINDOW+OUTPUT_H

X = np.array(X)
y = np.array(y)

print(f"\n✓ Sequences created:")
print(f"  X shape: {X.shape} (samples, timesteps, features)")
print(f"  y shape: {y.shape} (samples, horizon)")
```

```
================================================================================
STEP 4: SEQUENCE CREATION
================================================================================

Sequence Configuration:
  Input window: 72 timesteps = 6.0 hours
  Output horizon: 12 timesteps = 60 minutes

✓ Features: 50
✓ Target: Flow

✓ Sequences created:
  X shape: (34715, 72, 50) (samples, timesteps, features)
  y shape: (34715, 12) (samples, horizon)
```

Time-based Split

```
print("\n" + "="*80)
print("STEP 5: TIME-BASED SPLIT")
print("="*80)

# Split ratios
TRAIN_RATIO = 0.70
VAL_RATIO = 0.15
TEST_RATIO = 0.15
```

```
n_samples = len(X)
train_end = int(n_samples * TRAIN_RATIO)
val_end = int(n_samples * (TRAIN_RATIO + VAL_RATIO))

# Perform split
X_train = X[:train_end]
y_train = y[:train_end]
X_val = X[train_end:val_end]
y_val = y[train_end:val_end]
X_test = X[val_end:]
y_test = y[val_end:]

print(f"\n✓ Split complete:")
print(f"  Train: {len(X_train):,} samples (70%)")
print(f"  Val: {len(X_val):,} samples (15%)")
print(f"  Test: {len(X_test):,} samples (15%)")
```

```
================================================================================
STEP 5: TIME-BASED SPLIT
================================================================================

✓ Split complete:
  Train: 24,300 samples (70%)
  Val: 5,207 samples (15%)
  Test: 5,208 samples (15%)
```

Scaling

```
print("\n" + "="*80)
print("STEP 6: SCALING (NO DATA LEAKAGE)")
print("="*80)

# Scale features
scaler_X = RobustScaler()
X_train_reshaped = X_train.reshape(-1, X_train.shape[2])
scaler_X.fit(X_train_reshaped)

X_train_scaled = scaler_X.transform(X_train.reshape(-1, X_train.shape[2])).r
X_val_scaled = scaler_X.transform(X_val.reshape(-1, X_val.shape[2])).reshape
X_test_scaled = scaler_X.transform(X_test.reshape(-1, X_test.shape[2])).resh

# Scale target
scaler_y = RobustScaler()
scaler_y.fit(y_train)

y_train_scaled = scaler_y.transform(y_train)
y_val_scaled = scaler_y.transform(y_val)
y_test_scaled = scaler_y.transform(y_test)

print(f"\n✓ Scaling complete")
print(f"  X mean: {X_train_scaled.mean():.6f} (should be ~0)")
```

```
print(f"  X std: {X_train_scaled.std():.6f} (should be ~1)")
```

```
================================================================================
STEP 6: SCALING (NO DATA LEAKAGE)
================================================================================

✓ Scaling complete
  X mean: 0.045615 (should be ~0)
  X std: 0.971921 (should be ~1)
```

Baseline Model - Persistence

```
print("\n" + "="*80)
print("STEP 7: BASELINE MODEL")
print("="*80)

# Find flow lag feature
try:
    flow_lag_idx = feature_cols.index('flow_lag_3')
except:
    flow_lag_features = [i for i, col in enumerate(feature_cols) if 'flow_la
    flow_lag_idx = flow_lag_features[0]

# Get unscaled test data
X_test_unscaled = scaler_X.inverse_transform(
    X_test_scaled.reshape(-1, X_test_scaled.shape[2])
).reshape(X_test_scaled.shape)

# Predict: repeat last known value
last_known_flow = X_test_unscaled[:, -1, flow_lag_idx]
y_pred_baseline = np.tile(last_known_flow.reshape(-1, 1), (1, OUTPUT_HORIZON

# Evaluate
mae_baseline = mean_absolute_error(y_test, y_pred_baseline)
rmse_baseline = np.sqrt(mean_squared_error(y_test, y_pred_baseline))
r2_baseline = r2_score(y_test.flatten(), y_pred_baseline.flatten())
mape_baseline = np.mean(np.abs((y_test - y_pred_baseline) / y_test)) * 100

# Step-wise MAE
step_maes_baseline = [mean_absolute_error(y_test[:, step], y_pred_baseline[:
                      for step in range(OUTPUT_HORIZON)]

print(f"\n✓ Baseline (Persistence) Results:")
print(f"  MAE: {mae_baseline:.2f} vehicles")
print(f"  RMSE: {rmse_baseline:.2f} vehicles")
print(f"  R²: {r2_baseline:.4f}")
print(f"  MAPE: {mape_baseline:.2f}%")
```

```
================================================================================
STEP 7: BASELINE MODEL
================================================================================
```

```
✓ Baseline (Persistence) Results:
  MAE: 44.21 vehicles
  RMSE: 58.71 vehicles
  R²: 0.8853
  MAPE: 12.33%
```

Model Building Functions

```
print("\n" + "="*80)
print("STEP 8: MODEL DEFINITIONS")
print("="*80)

def build_gru():
    """GRU model for traffic prediction"""
    model = Sequential([
        GRU(128, input_shape=(INPUT_WINDOW, X_train_scaled.shape[2])),
        Dropout(0.3),
        Dense(OUTPUT_HORIZON)
    ])
    model.compile(optimizer=Adam(0.001), loss=tf.keras.losses.Huber(), metri
    return model

def build_lstm():
    """LSTM model for traffic prediction"""
    model = Sequential([
        LSTM(128, input_shape=(INPUT_WINDOW, X_train_scaled.shape[2])),
        Dropout(0.3),
        Dense(OUTPUT_HORIZON)
    ])
    model.compile(optimizer=Adam(0.001), loss=tf.keras.losses.Huber(), metri
    return model

def build_cnn_gru():
    """CNN+GRU hybrid model for traffic prediction"""
    model = Sequential([
        Conv1D(64, 3, activation='relu', padding='same',
               input_shape=(INPUT_WINDOW, X_train_scaled.shape[2])),
        MaxPooling1D(2),
        GRU(128),
        Dropout(0.3),
        Dense(64, activation='relu'),
        Dense(OUTPUT_HORIZON)
    ])
    model.compile(optimizer=Adam(0.001), loss=tf.keras.losses.Huber(), metri
    return model

def build_lstm_gru():
    """LSTM+GRU hybrid model for traffic prediction"""
    model = Sequential([
        LSTM(128, return_sequences=True, input_shape=(INPUT_WINDOW, X_train_
        Dropout(0.2),
        GRU(128),
```

```
        Dropout(0.3),
        Dense(64, activation='relu'),
        Dense(OUTPUT_HORIZON)
    ])
    model.compile(optimizer=Adam(0.001), loss=tf.keras.losses.Huber(), metri
    return model

print("\n✓ Model functions defined:")
print("  1. GRU")
print("  2. LSTM")
print("  3. CNN+GRU")
print("  4. LSTM+GRU")

# Early stopping
early_stopping = EarlyStopping(monitor='val_loss', patience=5, restore_best_
```

```
================================================================================
STEP 8: MODEL DEFINITIONS
================================================================================

✓ Model functions defined:
  1. GRU
  2. LSTM
  3. CNN+GRU
  4. LSTM+GRU
```

```
# ============================================================================
# HYPERPARAMETER TUNING SECTION (Optional - Insert after CELL 17)
# This demonstrates systematic hyperparameter optimization
# ============================================================================

# ============================================================================
# CELL 17A: Hyperparameter Tuning Setup
# ============================================================================

print("\n" + "="*80)
print("STEP 8A: HYPERPARAMETER TUNING (OPTIONAL)")
print("="*80)

print("\n⚙️ Testing different hyperparameter configurations...")
print("This may take 15-30 minutes depending on your hardware.\n")

# Hyperparameter search space
param_grid = {
    'learning_rate': [0.001, 0.0001],
    'batch_size': [32, 64],
    'dropout_rate': [0.2, 0.3],
    'hidden_units': [64, 128]
}

print("Search Space:")
print(f"  • Learning rates: {param_grid['learning_rate']}")
```

```
print(f"  • Batch sizes: {param_grid['batch_size']}")
print(f"  • Dropout rates: {param_grid['dropout_rate']}")
print(f"  • Hidden units: {param_grid['hidden_units']}")
print(f"\nTotal configurations to test: {len(param_grid['learning_rate']) *


# ============================================================================
# CELL 17B: Hyperparameter Tuning - Build Function
# ============================================================================

def build_model_tuning(learning_rate, dropout_rate, hidden_units):
    """
    Build model with configurable hyperparameters for tuning
    """
    model = Sequential([
        GRU(hidden_units, input_shape=(INPUT_WINDOW, X_train_scaled.shape[2]
        Dropout(dropout_rate),
        Dense(OUTPUT_HORIZON)
    ])

    model.compile(
        optimizer=Adam(learning_rate),
        loss=tf.keras.losses.Huber(),
        metrics=['mae']
    )

    return model


# ============================================================================
# CELL 17C: Hyperparameter Tuning - Grid Search
# ============================================================================

print("\n🔍 Starting grid search...")

# Store results
tuning_results = []

# Early stopping for tuning
early_stop_tuning = EarlyStopping(
    monitor='val_loss',
    patience=3,  # Reduced patience for faster tuning
    restore_best_weights=True,
    verbose=0
)

config_num = 0
total_configs = len(param_grid['learning_rate']) * len(param_grid['batch_siz
                len(param_grid['dropout_rate']) * len(param_grid['hidden_uni

# Grid search
for lr in param_grid['learning_rate']:
    for batch_size in param_grid['batch_size']:
        for dropout in param_grid['dropout_rate']:
            for hidden in param_grid['hidden_units']:
```

```
                    config_num += 1

                    print(f"\n[{config_num}/{total_configs}] Testing: LR={lr}, B

                    # Build model
                    model = build_model_tuning(lr, dropout, hidden)

                    # Train (reduced epochs for tuning)
                    history = model.fit(
                        X_train_scaled, y_train_scaled,
                        validation_data=(X_val_scaled, y_val_scaled),
                        epochs=20,  # Reduced epochs for faster tuning
                        batch_size=batch_size,
                        callbacks=[early_stop_tuning],
                        verbose=0
                    )

                    # Get best validation loss
                    best_val_loss = min(history.history['val_loss'])
                    best_val_mae = min(history.history['val_mae'])

                    # Store results
                    tuning_results.append({
                        'learning_rate': lr,
                        'batch_size': batch_size,
                        'dropout_rate': dropout,
                        'hidden_units': hidden,
                        'val_loss': best_val_loss,
                        'val_mae': best_val_mae
                    })

                    print(f"  ✓ Val Loss: {best_val_loss:.4f}, Val MAE: {best_va

print("\n✓ Grid search complete!")


# ============================================================================
# CELL 17D: Hyperparameter Tuning - Results Analysis
# ============================================================================

print("\n" + "="*80)
print("HYPERPARAMETER TUNING RESULTS")
print("="*80)

# Create results dataframe
tuning_df = pd.DataFrame(tuning_results)
tuning_df = tuning_df.sort_values('val_mae')

print("\nTop 5 Configurations (by Validation MAE):")
print(tuning_df.head(5).to_string(index=False))

# Best configuration
best_config = tuning_df.iloc[0]

print("\n🏆 BEST CONFIGURATION:")
```

```
print(f"  • Learning Rate: {best_config['learning_rate']}")
print(f"  • Batch Size: {int(best_config['batch_size'])}")
print(f"  • Dropout Rate: {best_config['dropout_rate']}")
print(f"  • Hidden Units: {int(best_config['hidden_units'])}")
print(f"  • Validation Loss: {best_config['val_loss']:.4f}")
print(f"  • Validation MAE: {best_config['val_mae']:.4f}")

# Save tuning results
tuning_df.to_csv('hyperparameter_tuning_results.csv', index=False)
print("\n✓ Tuning results saved to: hyperparameter_tuning_results.csv")


# ============================================================================
# CELL 17E: Visualize Hyperparameter Tuning Results
# ============================================================================

print("\n📊 Creating hyperparameter tuning visualizations...")

fig, axes = plt.subplots(2, 2, figsize=(16, 12))

# 1. Learning Rate Impact
lr_grouped = tuning_df.groupby('learning_rate')['val_mae'].mean()
axes[0, 0].bar(range(len(lr_grouped)), lr_grouped.values, color='steelblue',
axes[0, 0].set_xticks(range(len(lr_grouped)))
axes[0, 0].set_xticklabels([f'{lr}' for lr in lr_grouped.index])
axes[0, 0].set_xlabel('Learning Rate', fontsize=12)
axes[0, 0].set_ylabel('Average Validation MAE', fontsize=12)
axes[0, 0].set_title('Impact of Learning Rate', fontsize=13, fontweight='bol
axes[0, 0].grid(True, alpha=0.3, axis='y')

# 2. Batch Size Impact
batch_grouped = tuning_df.groupby('batch_size')['val_mae'].mean()
axes[0, 1].bar(range(len(batch_grouped)), batch_grouped.values, color='coral
axes[0, 1].set_xticks(range(len(batch_grouped)))
axes[0, 1].set_xticklabels([f'{int(bs)}' for bs in batch_grouped.index])
axes[0, 1].set_xlabel('Batch Size', fontsize=12)
axes[0, 1].set_ylabel('Average Validation MAE', fontsize=12)
axes[0, 1].set_title('Impact of Batch Size', fontsize=13, fontweight='bold')
axes[0, 1].grid(True, alpha=0.3, axis='y')

# 3. Dropout Rate Impact
dropout_grouped = tuning_df.groupby('dropout_rate')['val_mae'].mean()
axes[1, 0].bar(range(len(dropout_grouped)), dropout_grouped.values, color='m
axes[1, 0].set_xticks(range(len(dropout_grouped)))
axes[1, 0].set_xticklabels([f'{dr}' for dr in dropout_grouped.index])
axes[1, 0].set_xlabel('Dropout Rate', fontsize=12)
axes[1, 0].set_ylabel('Average Validation MAE', fontsize=12)
axes[1, 0].set_title('Impact of Dropout Rate', fontsize=13, fontweight='bold
axes[1, 0].grid(True, alpha=0.3, axis='y')

# 4. Hidden Units Impact
hidden_grouped = tuning_df.groupby('hidden_units')['val_mae'].mean()
axes[1, 1].bar(range(len(hidden_grouped)), hidden_grouped.values, color='med
axes[1, 1].set_xticks(range(len(hidden_grouped)))
axes[1, 1].set_xticklabels([f'{int(hu)}' for hu in hidden_grouped.index])
```

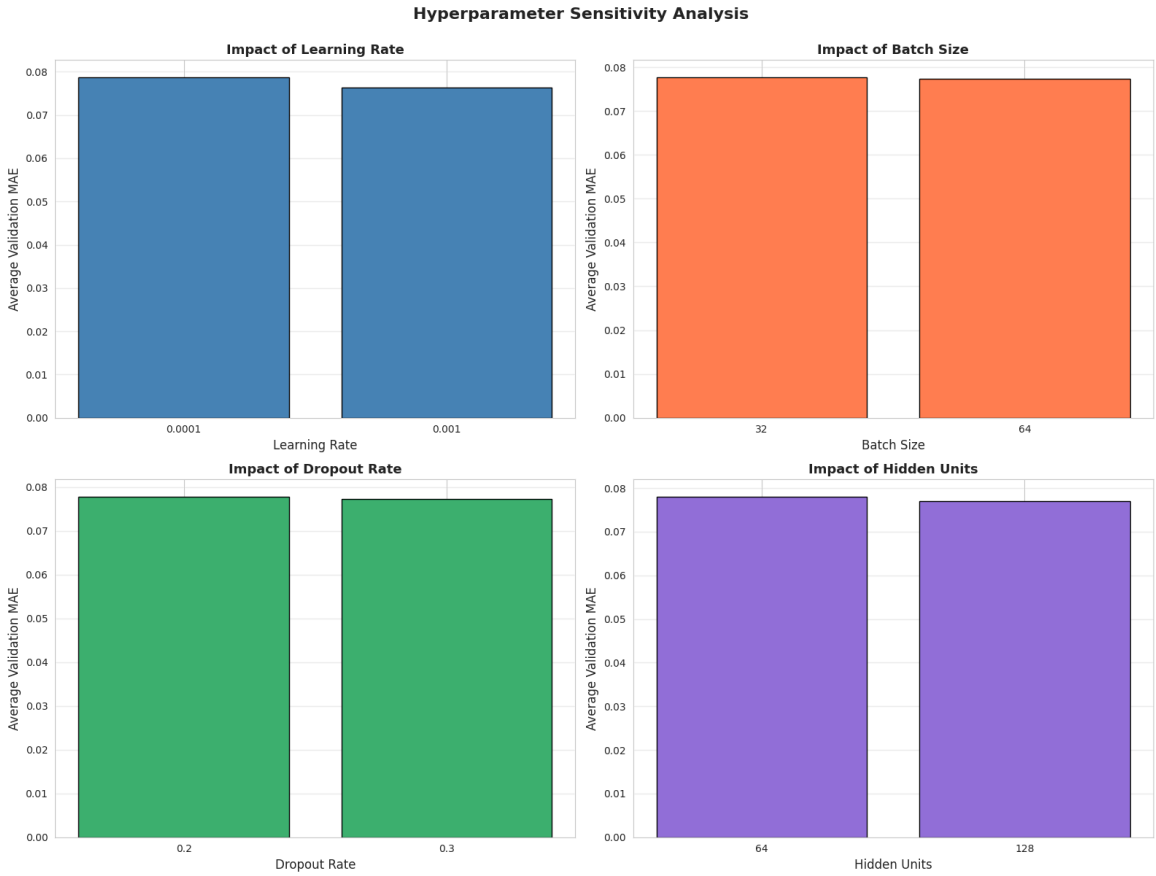
```
axes[1, 1].set_xlabel('Hidden Units', fontsize=12)
axes[1, 1].set_ylabel('Average Validation MAE', fontsize=12)
axes[1, 1].set_title('Impact of Hidden Units', fontsize=13, fontweight='bold
axes[1, 1].grid(True, alpha=0.3, axis='y')

plt.suptitle('Hyperparameter Sensitivity Analysis', fontsize=16, fontweight=
plt.tight_layout()
plt.savefig('TUNING_hyperparameter_impact.png', dpi=300, bbox_inches='tight'
print("✓ Saved: TUNING_hyperparameter_impact.png")
plt.show()


# ============================================================================
# CELL 17F: Update Model Functions with Best Hyperparameters
# ============================================================================

print("\n⚙️ Updating model functions with optimized hyperparameters...")

# Extract best hyperparameters
BEST_LR = best_config['learning_rate']
BEST_BATCH_SIZE = int(best_config['batch_size'])
BEST_DROPOUT = best_config['dropout_rate']
BEST_HIDDEN = int(best_config['hidden_units'])

def build_gru_optimized():
    """GRU model with optimized hyperparameters from tuning"""
    model = Sequential([
        GRU(BEST_HIDDEN, input_shape=(INPUT_WINDOW, X_train_scaled.shape[2])
        Dropout(BEST_DROPOUT),
        Dense(OUTPUT_HORIZON)
    ])
    model.compile(optimizer=Adam(BEST_LR), loss=tf.keras.losses.Huber(), met
    return model

def build_lstm_optimized():
    """LSTM model with optimized hyperparameters from tuning"""
    model = Sequential([
        LSTM(BEST_HIDDEN, input_shape=(INPUT_WINDOW, X_train_scaled.shape[2]
        Dropout(BEST_DROPOUT),
        Dense(OUTPUT_HORIZON)
    ])
    model.compile(optimizer=Adam(BEST_LR), loss=tf.keras.losses.Huber(), met
    return model

def build_cnn_gru_optimized():
    """CNN+GRU model with optimized hyperparameters from tuning"""
    model = Sequential([
        Conv1D(64, 3, activation='relu', padding='same',
               input_shape=(INPUT_WINDOW, X_train_scaled.shape[2])),
        MaxPooling1D(2),
        GRU(BEST_HIDDEN),
        Dropout(BEST_DROPOUT),
        Dense(64, activation='relu'),
        Dense(OUTPUT_HORIZON)
    ])
```

```
    model.compile(optimizer=Adam(BEST_LR), loss=tf.keras.losses.Huber(), met
    return model

def build_lstm_gru_optimized():
    """LSTM+GRU model with optimized hyperparameters from tuning"""
    model = Sequential([
        LSTM(BEST_HIDDEN, return_sequences=True, input_shape=(INPUT_WINDOW,
        Dropout(BEST_DROPOUT * 0.7),  # Slightly less dropout in first layer
        GRU(BEST_HIDDEN),
        Dropout(BEST_DROPOUT),
        Dense(64, activation='relu'),
        Dense(OUTPUT_HORIZON)
    ])
    model.compile(optimizer=Adam(BEST_LR), loss=tf.keras.losses.Huber(), met
    return model

print("\n✓ Model functions updated with optimal hyperparameters:")
print(f"  • Learning Rate: {BEST_LR}")
print(f"  • Batch Size: {BEST_BATCH_SIZE}")
print(f"  • Dropout Rate: {BEST_DROPOUT}")
print(f"  • Hidden Units: {BEST_HIDDEN}")
print("\nNote: Use these optimized functions (build_xxx_optimized) instead o
print("      functions when training your final models.")

# ==========================================================================
# IMPORTANT NOTE:
# After running this hyperparameter tuning section, replace the original
# model building functions (build_gru, build_lstm, etc.) with the optimized
# versions (build_gru_optimized, build_lstm_optimized, etc.) in subsequent c
# ==========================================================================
```

```
================================================================================
STEP 8A: HYPERPARAMETER TUNING (OPTIONAL)
================================================================================

⚙️ Testing different hyperparameter configurations...
This may take 15-30 minutes depending on your hardware.

Search Space:
  • Learning rates: [0.001, 0.0001]
  • Batch sizes: [32, 64]
  • Dropout rates: [0.2, 0.3]
  • Hidden units: [64, 128]

Total configurations to test: 16

🔍 Starting grid search...

[1/16] Testing: LR=0.001, Batch=32, Dropout=0.2, Hidden=64
  ✓ Val Loss: 0.0059, Val MAE: 0.0788

[2/16] Testing: LR=0.001, Batch=32, Dropout=0.2, Hidden=128
  ✓ Val Loss: 0.0056, Val MAE: 0.0761

[3/16] Testing: LR=0.001, Batch=32, Dropout=0.3, Hidden=64
  ✓ Val Loss: 0.0056, Val MAE: 0.0759

[4/16] Testing: LR=0.001, Batch=32, Dropout=0.3, Hidden=128
  ✓ Val Loss: 0.0058, Val MAE: 0.0774

[5/16] Testing: LR=0.001, Batch=64, Dropout=0.2, Hidden=64
  ✓ Val Loss: 0.0060, Val MAE: 0.0767

[6/16] Testing: LR=0.001, Batch=64, Dropout=0.2, Hidden=128
  ✓ Val Loss: 0.0056, Val MAE: 0.0753

[7/16] Testing: LR=0.001, Batch=64, Dropout=0.3, Hidden=64
  ✓ Val Loss: 0.0057, Val MAE: 0.0749

[8/16] Testing: LR=0.001, Batch=64, Dropout=0.3, Hidden=128
  ✓ Val Loss: 0.0057, Val MAE: 0.0754

[9/16] Testing: LR=0.0001, Batch=32, Dropout=0.2, Hidden=64
  ✓ Val Loss: 0.0065, Val MAE: 0.0819

[10/16] Testing: LR=0.0001, Batch=32, Dropout=0.2, Hidden=128
  ✓ Val Loss: 0.0060, Val MAE: 0.0772

[11/16] Testing: LR=0.0001, Batch=32, Dropout=0.3, Hidden=64
  ✓ Val Loss: 0.0062, Val MAE: 0.0785

[12/16] Testing: LR=0.0001, Batch=32, Dropout=0.3, Hidden=128
  ✓ Val Loss: 0.0058, Val MAE: 0.0759

[13/16] Testing: LR=0.0001, Batch=64, Dropout=0.2, Hidden=64
  ✓ Val Loss: 0.0062, Val MAE: 0.0788

[14/16] Testing: LR=0.0001, Batch=64, Dropout=0.2, Hidden=128
  ✓ Val Loss: 0.0060, Val MAE: 0.0780
```

```
[15/16] Testing: LR=0.0001, Batch=64, Dropout=0.3, Hidden=64
  ✓ Val Loss: 0.0062, Val MAE: 0.0791

[16/16] Testing: LR=0.0001, Batch=64, Dropout=0.3, Hidden=128
  ✓ Val Loss: 0.0063, Val MAE: 0.0807

✓ Grid search complete!

================================================================================
HYPERPARAMETER TUNING RESULTS
================================================================================

Top 5 Configurations (by Validation MAE):
 learning_rate  batch_size  dropout_rate  hidden_units  val_loss  val_mae
        0.0010          64           0.3            64  0.005657 0.074926
        0.0010          64           0.2           128  0.005609 0.075337
        0.0010          64           0.3           128  0.005659 0.075443
        0.0010          32           0.3            64  0.005560 0.075863
        0.0001          32           0.3           128  0.005786 0.075903

🏆 BEST CONFIGURATION:
  • Learning Rate: 0.001
  • Batch Size: 64
  • Dropout Rate: 0.3
  • Hidden Units: 64
  • Validation Loss: 0.0057
  • Validation MAE: 0.0749
```

```
✓ Tuning results saved to: hyperparameter_tuning_results.csv

📊 Creating hyperparameter tuning visualizations...
✓ Saved: TUNING_hyperparameter_impact.png
```




```
⚙️ Updating model functions with optimized hyperparameters...
```

```
✓ Model functions updated with optimal hyperparameters:
  • Learning Rate: 0.001
  • Batch Size: 64
```

Train GRU Model

```
print("\n" + "="*80)
print("MODEL 1: GRU")
print("="*80)

gru_model = build_gru_optimized()
print("\n✓ GRU Model Architecture:")
gru_model.summary()

print("\n🚀 Training GRU model...")
history_gru = gru_model.fit(
    X_train_scaled, y_train_scaled,
    validation_data=(X_val_scaled, y_val_scaled),
    epochs=40,
    batch_size=64,
    callbacks=[early_stopping],
    verbose=1
)
print("✓ GRU training complete")
```

```
================================================================================
MODEL 1: GRU
================================================================================

✓ GRU Model Architecture:
Model: "sequential_23"
```

Layer (type)	Output Shape	Param #
gru_21 (GRU)	(None, 64)	22,272
dropout_24 (Dropout)	(None, 64)	0
dense_26 (Dense)	(None, 12)	780

```
Total params: 23,052 (90.05 KB)
Trainable params: 23,052 (90.05 KB)
Non-trainable params: 0 (0.00 B)

🚀 Training GRU model...
Epoch 1/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - loss: 0.0691 - mae: 0.2670 - val_l
Epoch 2/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.0150 - mae: 0.1313 - val_l
Epoch 3/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - loss: 0.0119 - mae: 0.1162 - val_l
Epoch 4/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - loss: 0.0102 - mae: 0.1079 - val_l
Epoch 5/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.0090 - mae: 0.1007 - val_l
Epoch 6/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.0080 - mae: 0.0955 - val_l
Epoch 7/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - loss: 0.0072 - mae: 0.0906 - val_l
Epoch 8/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - loss: 0.0067 - mae: 0.0872 - val_l
Epoch 9/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.0066 - mae: 0.0861 - val_l
Epoch 10/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.0061 - mae: 0.0836 - val_l
Epoch 11/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.0059 - mae: 0.0822 - val_l
Epoch 12/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - loss: 0.0058 - mae: 0.0814 - val_l
Epoch 13/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.0056 - mae: 0.0805 - val_l
Epoch 13: early stopping
Restoring model weights from the end of the best epoch: 8.
```

Evaluate GRU Model

```
print("\n" + "="*80)
print("EVALUATING GRU MODEL")
print("="*80)

# Predict
```

```
y_pred_gru_scaled = gru_model.predict(X_test_scaled, verbose=0)
y_pred_gru = scaler_y.inverse_transform(y_pred_gru_scaled)

# Metrics
mae_gru = mean_absolute_error(y_test, y_pred_gru)
rmse_gru = np.sqrt(mean_squared_error(y_test, y_pred_gru))
r2_gru = r2_score(y_test.flatten(), y_pred_gru.flatten())
mape_gru = np.mean(np.abs((y_test - y_pred_gru) / y_test)) * 100

# Step-wise MAE
step_maes_gru = [mean_absolute_error(y_test[:, step], y_pred_gru[:, step])
                 for step in range(OUTPUT_HORIZON)]

print(f"\n✓ GRU Results:")
print(f"  MAE: {mae_gru:.2f} vehicles")
print(f"  RMSE: {rmse_gru:.2f} vehicles")
print(f"  R²: {r2_gru:.4f}")
print(f"  MAPE: {mape_gru:.2f}%")
print(f"  Improvement over baseline: {((mae_baseline - mae_gru) / mae_baseli
```

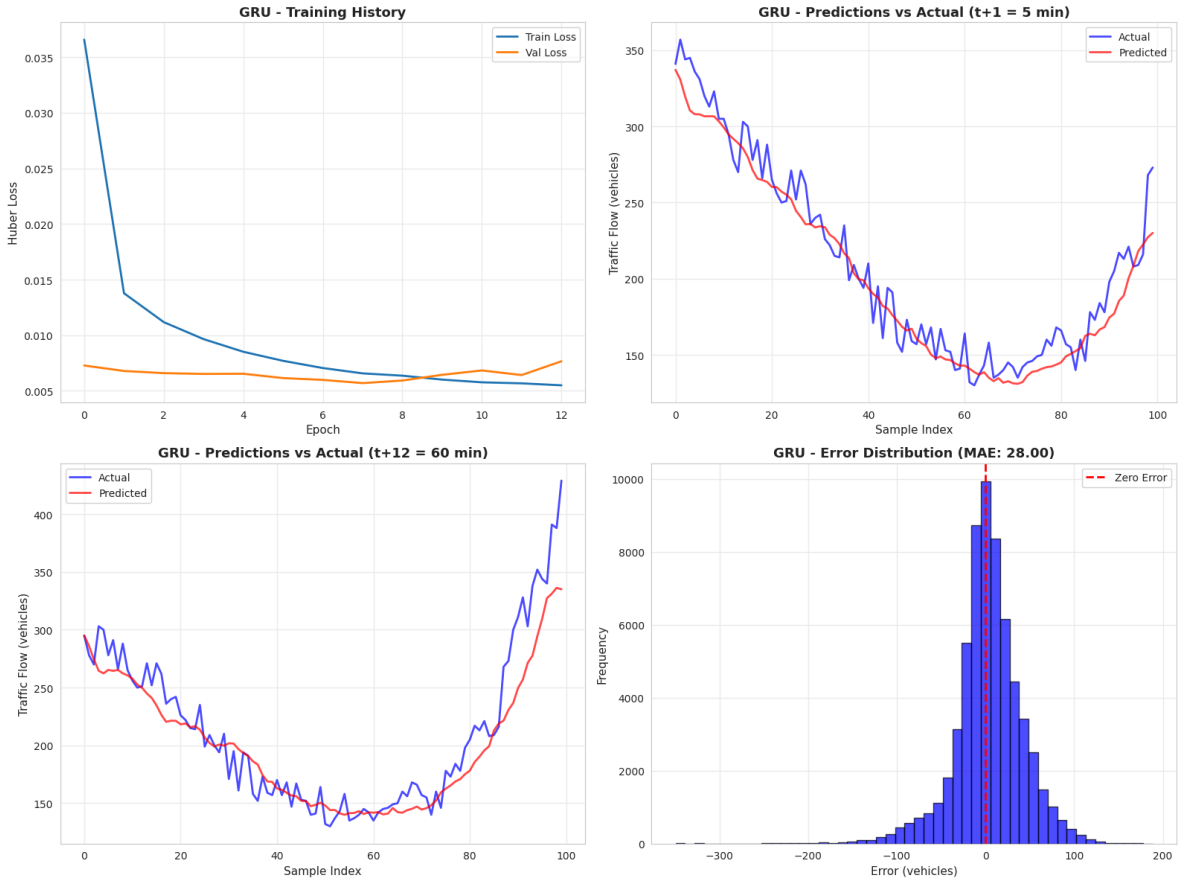
```
================================================================================
EVALUATING GRU MODEL
================================================================================

✓ GRU Results:
  MAE: 28.00 vehicles
  RMSE: 39.33 vehicles
  R²: 0.9485
  MAPE: 7.10%
  Improvement over baseline: 36.7%
```

Visualize GRU Results

```
print("\n📊 Generating GRU visualizations...")

fig, axes = plt.subplots(2, 2, figsize=(16, 12))

# Training history
axes[0, 0].plot(history_gru.history['loss'], label='Train Loss', linewidth=2
axes[0, 0].plot(history_gru.history['val_loss'], label='Val Loss', linewidth
axes[0, 0].set_xlabel('Epoch', fontsize=11)
axes[0, 0].set_ylabel('Huber Loss', fontsize=11)
axes[0, 0].set_title('GRU - Training History', fontsize=13, fontweight='bold
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

# Predictions vs Actual (t+1)
sample_size = 100
axes[0, 1].plot(y_test[:sample_size, 0], label='Actual', linewidth=2, color=
axes[0, 1].plot(y_pred_gru[:sample_size, 0], label='Predicted', linewidth=2,
axes[0, 1].set_xlabel('Sample Index', fontsize=11)
axes[0, 1].set_ylabel('Traffic Flow (vehicles)', fontsize=11)
```

```
axes[0, 1].set_title('GRU - Predictions vs Actual (t+1 = 5 min)', fontsize=1
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

# Predictions vs Actual (t+12)
axes[1, 0].plot(y_test[:sample_size, -1], label='Actual', linewidth=2, color
axes[1, 0].plot(y_pred_gru[:sample_size, -1], label='Predicted', linewidth=2
axes[1, 0].set_xlabel('Sample Index', fontsize=11)
axes[1, 0].set_ylabel('Traffic Flow (vehicles)', fontsize=11)
axes[1, 0].set_title('GRU - Predictions vs Actual (t+12 = 60 min)', fontsize
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)

# Error distribution
errors_gru = (y_test - y_pred_gru).flatten()
axes[1, 1].hist(errors_gru, bins=50, color='blue', edgecolor='black', alpha=
axes[1, 1].axvline(0, color='red', linestyle='--', linewidth=2, label='Zero
axes[1, 1].set_xlabel('Error (vehicles)', fontsize=11)
axes[1, 1].set_ylabel('Frequency', fontsize=11)
axes[1, 1].set_title(f'GRU - Error Distribution (MAE: {mae_gru:.2f})', fonts
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('GRU_complete_analysis.png', dpi=300, bbox_inches='tight')
print("✓ Saved: GRU_complete_analysis.png")
plt.show()
```

```
📊 Generating GRU visualizations...
✓ Saved: GRU_complete_analysis.png
```



Train LSTM Model

```
print("\n" + "="*80)
print("MODEL 2: LSTM")
print("="*80)

lstm_model = build_lstm_optimized()
print("\n✓ LSTM Model Architecture:")
lstm_model.summary()
```

```
print("\n🚀 Training LSTM model...")
history_lstm = lstm_model.fit(
    X_train_scaled, y_train_scaled,
    validation_data=(X_val_scaled, y_val_scaled),
    epochs=40,
    batch_size=64,
    callbacks=[early_stopping],
    verbose=1
)
print("✓ LSTM training complete")
```

```
================================================================================
MODEL 2: LSTM
================================================================================

✓ LSTM Model Architecture:
Model: "sequential_24"
```

Layer (type)	Output Shape	Param #
lstm_3 (LSTM)	(None, 64)	29,440
dropout_25 (Dropout)	(None, 64)	0
dense_27 (Dense)	(None, 12)	780

```
 Total params: 30,220 (118.05 KB)
 Trainable params: 30,220 (118.05 KB)
 Non-trainable params: 0 (0.00 B)

🚀 Training LSTM model...
Epoch 1/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - loss: 0.0436 - mae: 0.2160 - val_
Epoch 2/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.0117 - mae: 0.1164 - val_l
Epoch 3/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.0088 - mae: 0.1009 - val_l
Epoch 4/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.0077 - mae: 0.0944 - val_l
Epoch 5/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - loss: 0.0070 - mae: 0.0899 - val_l
Epoch 6/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.0067 - mae: 0.0873 - val_l
Epoch 7/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 5s 7ms/step - loss: 0.0062 - mae: 0.0845 - val_l
Epoch 8/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - loss: 0.0060 - mae: 0.0828 - val_l
Epoch 8: early stopping
Restoring model weights from the end of the best epoch: 3.
```

Evaluate LSTM Model

```
print("\n" + "="*80)
print("EVALUATING LSTM MODEL")
print("="*80)
```

```
# Predict
y_pred_lstm_scaled = lstm_model.predict(X_test_scaled, verbose=0)
y_pred_lstm = scaler_y.inverse_transform(y_pred_lstm_scaled)

# Metrics
mae_lstm = mean_absolute_error(y_test, y_pred_lstm)
rmse_lstm = np.sqrt(mean_squared_error(y_test, y_pred_lstm))
r2_lstm = r2_score(y_test.flatten(), y_pred_lstm.flatten())
mape_lstm = np.mean(np.abs((y_test - y_pred_lstm) / y_test)) * 100

# Step-wise MAE
step_maes_lstm = [mean_absolute_error(y_test[:, step], y_pred_lstm[:, step])
                  for step in range(OUTPUT_HORIZON)]

print(f"\n✓ LSTM Results:")
print(f"  MAE: {mae_lstm:.2f} vehicles")
print(f"  RMSE: {rmse_lstm:.2f} vehicles")
print(f"  R²: {r2_lstm:.4f}")
print(f"  MAPE: {mape_lstm:.2f}%")
print(f"  Improvement over baseline: {((mae_baseline - mae_lstm) / mae_basel
```

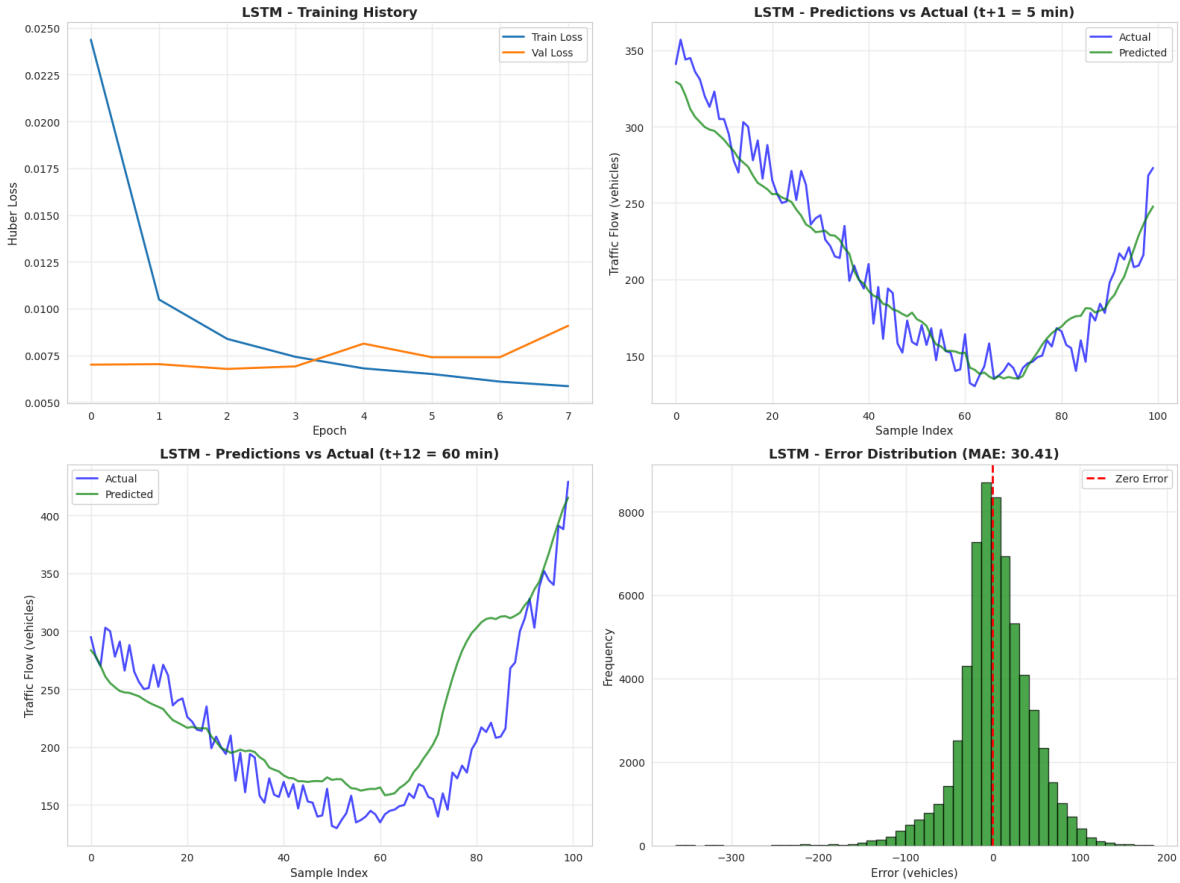
```
================================================================================
EVALUATING LSTM MODEL
================================================================================

✓ LSTM Results:
  MAE: 30.41 vehicles
  RMSE: 41.81 vehicles
  R²: 0.9418
  MAPE: 8.16%
  Improvement over baseline: 31.2%
```

Visualize LSTM Results

```
print("\n📊 Generating LSTM visualizations...")

fig, axes = plt.subplots(2, 2, figsize=(16, 12))

# Training history
axes[0, 0].plot(history_lstm.history['loss'], label='Train Loss', linewidth=
axes[0, 0].plot(history_lstm.history['val_loss'], label='Val Loss', linewidt
axes[0, 0].set_xlabel('Epoch', fontsize=11)
axes[0, 0].set_ylabel('Huber Loss', fontsize=11)
axes[0, 0].set_title('LSTM - Training History', fontsize=13, fontweight='bol
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

# Predictions vs Actual (t+1)
axes[0, 1].plot(y_test[:sample_size, 0], label='Actual', linewidth=2, color=
axes[0, 1].plot(y_pred_lstm[:sample_size, 0], label='Predicted', linewidth=2
axes[0, 1].set_xlabel('Sample Index', fontsize=11)
```

```
axes[0, 1].set_ylabel('Traffic Flow (vehicles)', fontsize=11)
axes[0, 1].set_title('LSTM - Predictions vs Actual (t+1 = 5 min)', fontsize=
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

# Predictions vs Actual (t+12)
axes[1, 0].plot(y_test[:sample_size, -1], label='Actual', linewidth=2, color
axes[1, 0].plot(y_pred_lstm[:sample_size, -1], label='Predicted', linewidth=
axes[1, 0].set_xlabel('Sample Index', fontsize=11)
axes[1, 0].set_ylabel('Traffic Flow (vehicles)', fontsize=11)
axes[1, 0].set_title('LSTM - Predictions vs Actual (t+12 = 60 min)', fontsiz
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)

# Error distribution
errors_lstm = (y_test - y_pred_lstm).flatten()
axes[1, 1].hist(errors_lstm, bins=50, color='green', edgecolor='black', alph
axes[1, 1].axvline(0, color='red', linestyle='--', linewidth=2, label='Zero
axes[1, 1].set_xlabel('Error (vehicles)', fontsize=11)
axes[1, 1].set_ylabel('Frequency', fontsize=11)
axes[1, 1].set_title(f'LSTM - Error Distribution (MAE: {mae_lstm:.2f})', fon
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('LSTM_complete_analysis.png', dpi=300, bbox_inches='tight')
print("✓ Saved: LSTM_complete_analysis.png")
plt.show()
```

```
📊 Generating LSTM visualizations...
✓ Saved: LSTM_complete_analysis.png
```



Train CNN+GRU Model

```
print("\n" + "="*80)
print("MODEL 3: CNN+GRU")
print("="*80)

cnn_gru_model = build_cnn_gru_optimized()
print("\n✓ CNN+GRU Model Architecture:")
cnn_gru_model.summary()
```

```
print("\n🚀 Training CNN+GRU model...")
history_cnn_gru = cnn_gru_model.fit(
    X_train_scaled, y_train_scaled,
    validation_data=(X_val_scaled, y_val_scaled),
    epochs=40,
    batch_size=64,
    callbacks=[early_stopping],
    verbose=1
)
print("✓ CNN+GRU training complete")
```

```
================================================================================
MODEL 3: CNN+GRU
================================================================================

✓ CNN+GRU Model Architecture:
Model: "sequential_25"
```

Layer (type)	Output Shape	Param #
conv1d_2 (Conv1D)	(None, 72, 64)	9,664
max_pooling1d_2 (MaxPooling1D)	(None, 36, 64)	0
gru_22 (GRU)	(None, 64)	24,960
dropout_26 (Dropout)	(None, 64)	0
dense_28 (Dense)	(None, 64)	4,160
dense_29 (Dense)	(None, 12)	780

```
 Total params: 39,564 (154.55 KB)
 Trainable params: 39,564 (154.55 KB)
 Non-trainable params: 0 (0.00 B)

🚀 Training CNN+GRU model...
Epoch 1/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 5s 8ms/step - loss: 0.0401 - mae: 0.2022 - val_l
Epoch 2/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 4s 10ms/step - loss: 0.0107 - mae: 0.1094 - val_
Epoch 3/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.0083 - mae: 0.0964 - val_l
Epoch 4/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.0072 - mae: 0.0899 - val_l
Epoch 5/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.0065 - mae: 0.0854 - val_l
Epoch 6/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - loss: 0.0058 - mae: 0.0813 - val_l
Epoch 7/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.0054 - mae: 0.0782 - val_l
Epoch 8/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.0051 - mae: 0.0764 - val_l
Epoch 9/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.0049 - mae: 0.0743 - val_l
Epoch 10/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 4s 10ms/step - loss: 0.0047 - mae: 0.0728 - val_
Epoch 11/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - loss: 0.0045 - mae: 0.0710 - val_l
Epoch 12/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - loss: 0.0043 - mae: 0.0695 - val_l
Epoch 13/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.0041 - mae: 0.0683 - val_l
Epoch 13: early stopping
Restoring model weights from the end of the best epoch: 8.
```

Evaluate CNN+GRU Model

```
print("\n" + "="*80)
print("EVALUATING CNN+GRU MODEL")
print("="*80)

# Predict
y_pred_cnn_gru_scaled = cnn_gru_model.predict(X_test_scaled, verbose=0)
y_pred_cnn_gru = scaler_y.inverse_transform(y_pred_cnn_gru_scaled)

# Metrics
mae_cnn_gru = mean_absolute_error(y_test, y_pred_cnn_gru)
rmse_cnn_gru = np.sqrt(mean_squared_error(y_test, y_pred_cnn_gru))
r2_cnn_gru = r2_score(y_test.flatten(), y_pred_cnn_gru.flatten())
mape_cnn_gru = np.mean(np.abs((y_test - y_pred_cnn_gru) / y_test)) * 100

# Step-wise MAE
step_maes_cnn_gru = [mean_absolute_error(y_test[:, step], y_pred_cnn_gru[:,
                     for step in range(OUTPUT_HORIZON)]

print(f"\n✓ CNN+GRU Results:")
print(f"  MAE: {mae_cnn_gru:.2f} vehicles")
print(f"  RMSE: {rmse_cnn_gru:.2f} vehicles")
print(f"  R²: {r2_cnn_gru:.4f}")
print(f"  MAPE: {mape_cnn_gru:.2f}%")
print(f"  Improvement over baseline: {((mae_baseline - mae_cnn_gru) / mae_ba
```

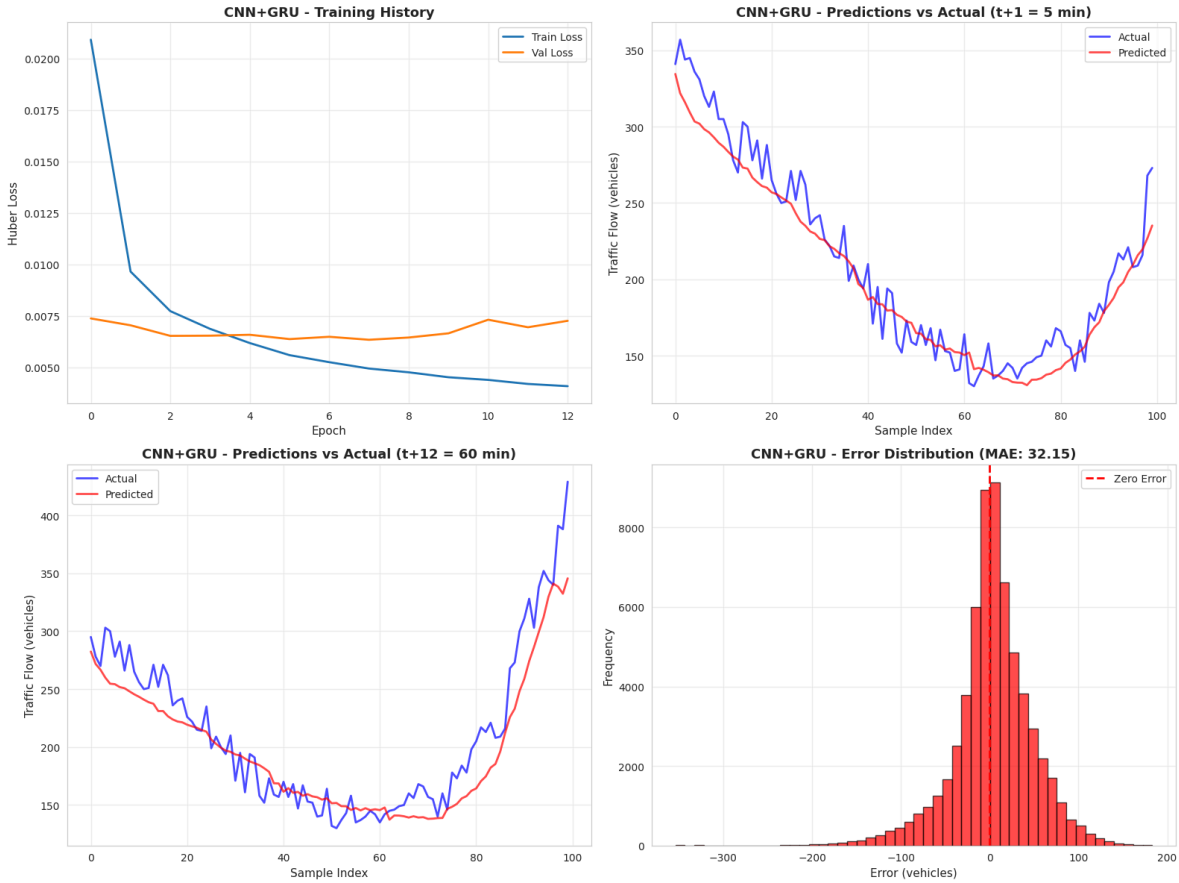
```
================================================================================
EVALUATING CNN+GRU MODEL
================================================================================

✓ CNN+GRU Results:
  MAE: 32.15 vehicles
  RMSE: 45.11 vehicles
  R²: 0.9323
  MAPE: 7.95%
  Improvement over baseline: 27.3%
```

Visualize CNN+GRU Results

```
print("\n📊 Generating CNN+GRU visualizations...")

fig, axes = plt.subplots(2, 2, figsize=(16, 12))

# Training history
axes[0, 0].plot(history_cnn_gru.history['loss'], label='Train Loss', linewid
axes[0, 0].plot(history_cnn_gru.history['val_loss'], label='Val Loss', linew
axes[0, 0].set_xlabel('Epoch', fontsize=11)
axes[0, 0].set_ylabel('Huber Loss', fontsize=11)
axes[0, 0].set_title('CNN+GRU - Training History', fontsize=13, fontweight='
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

# Predictions vs Actual (t+1)
axes[0, 1].plot(y_test[:sample_size, 0], label='Actual', linewidth=2, color=
```

```
axes[0, 1].plot(y_pred_cnn_gru[:sample_size, 0], label='Predicted', linewidt
axes[0, 1].set_xlabel('Sample Index', fontsize=11)
axes[0, 1].set_ylabel('Traffic Flow (vehicles)', fontsize=11)
axes[0, 1].set_title('CNN+GRU - Predictions vs Actual (t+1 = 5 min)', fontsi
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

# Predictions vs Actual (t+12)
axes[1, 0].plot(y_test[:sample_size, -1], label='Actual', linewidth=2, color
axes[1, 0].plot(y_pred_cnn_gru[:sample_size, -1], label='Predicted', linewid
axes[1, 0].set_xlabel('Sample Index', fontsize=11)
axes[1, 0].set_ylabel('Traffic Flow (vehicles)', fontsize=11)
axes[1, 0].set_title('CNN+GRU - Predictions vs Actual (t+12 = 60 min)', font
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)

# Error distribution
errors_cnn_gru = (y_test - y_pred_cnn_gru).flatten()
axes[1, 1].hist(errors_cnn_gru, bins=50, color='red', edgecolor='black', alp
axes[1, 1].axvline(0, color='red', linestyle='--', linewidth=2, label='Zero
axes[1, 1].set_xlabel('Error (vehicles)', fontsize=11)
axes[1, 1].set_ylabel('Frequency', fontsize=11)
axes[1, 1].set_title(f'CNN+GRU - Error Distribution (MAE: {mae_cnn_gru:.2f})
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('CNN_GRU_complete_analysis.png', dpi=300, bbox_inches='tight')
print("✓ Saved: CNN_GRU_complete_analysis.png")
plt.show()
```

```
📊 Generating CNN+GRU visualizations...
✓ Saved: CNN_GRU_complete_analysis.png
```



Train LSTM+GRU Model

```
print("\n" + "="*80)
print("MODEL 4: LSTM+GRU HYBRID (NEW!)")
print("="*80)

lstm_gru_model = build_lstm_gru_optimized()
print("\n✓ LSTM+GRU Model Architecture:")
lstm_gru_model.summary()
```

```
print("\n🚀 Training LSTM+GRU model...")
history_lstm_gru = lstm_gru_model.fit(
    X_train_scaled, y_train_scaled,
    validation_data=(X_val_scaled, y_val_scaled),
    epochs=40,
    batch_size=64,
    callbacks=[early_stopping],
    verbose=1
)
print("✓ LSTM+GRU training complete")
```

```
================================================================================
MODEL 4: LSTM+GRU HYBRID (NEW!)
================================================================================

✓ LSTM+GRU Model Architecture:
Model: "sequential_26"
```

Layer (type)	Output Shape	Param #
lstm_4 (LSTM)	(None, 72, 64)	29,440
dropout_27 (Dropout)	(None, 72, 64)	0
gru_23 (GRU)	(None, 64)	24,960
dropout_28 (Dropout)	(None, 64)	0
dense_30 (Dense)	(None, 64)	4,160
dense_31 (Dense)	(None, 12)	780

```
 Total params: 59,340 (231.80 KB)
 Trainable params: 59,340 (231.80 KB)
 Non-trainable params: 0 (0.00 B)

🚀 Training LSTM+GRU model...
Epoch 1/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 7s 12ms/step - loss: 0.0374 - mae: 0.1973 - val_
Epoch 2/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 5s 13ms/step - loss: 0.0093 - mae: 0.1033 - val_
Epoch 3/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step - loss: 0.0074 - mae: 0.0921 - val_
Epoch 4/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step - loss: 0.0063 - mae: 0.0849 - val_
Epoch 5/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 5s 13ms/step - loss: 0.0056 - mae: 0.0796 - val_
Epoch 6/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step - loss: 0.0051 - mae: 0.0762 - val_
Epoch 7/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 6s 13ms/step - loss: 0.0048 - mae: 0.0735 - val_
Epoch 8/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step - loss: 0.0045 - mae: 0.0715 - val_
Epoch 9/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step - loss: 0.0044 - mae: 0.0701 - val_
Epoch 10/40
380/380 ━━━━━━━━━━━━━━━━━━━━ 5s 13ms/step - loss: 0.0041 - mae: 0.0680 - val_
Epoch 10: early stopping
Restoring model weights from the end of the best epoch: 5.
```

Evaluate LSTM+GRU Model

```
print("\n" + "="*80)
print("EVALUATING LSTM+GRU MODEL")
print("="*80)

# Predict
```

```
y_pred_lstm_gru_scaled = lstm_gru_model.predict(X_test_scaled, verbose=0)
y_pred_lstm_gru = scaler_y.inverse_transform(y_pred_lstm_gru_scaled)

# Metrics
mae_lstm_gru = mean_absolute_error(y_test, y_pred_lstm_gru)
rmse_lstm_gru = np.sqrt(mean_squared_error(y_test, y_pred_lstm_gru))
r2_lstm_gru = r2_score(y_test.flatten(), y_pred_lstm_gru.flatten())
mape_lstm_gru = np.mean(np.abs((y_test - y_pred_lstm_gru) / y_test)) * 100

# Step-wise MAE
step_maes_lstm_gru = [mean_absolute_error(y_test[:, step], y_pred_lstm_gru[:
                      for step in range(OUTPUT_HORIZON)]

print(f"\n✓ LSTM+GRU Results:")
print(f"  MAE: {mae_lstm_gru:.2f} vehicles")
print(f"  RMSE: {rmse_lstm_gru:.2f} vehicles")
print(f"  R²: {r2_lstm_gru:.4f}")
print(f"  MAPE: {mape_lstm_gru:.2f}%")
print(f"  Improvement over baseline: {((mae_baseline - mae_lstm_gru) / mae_b
```

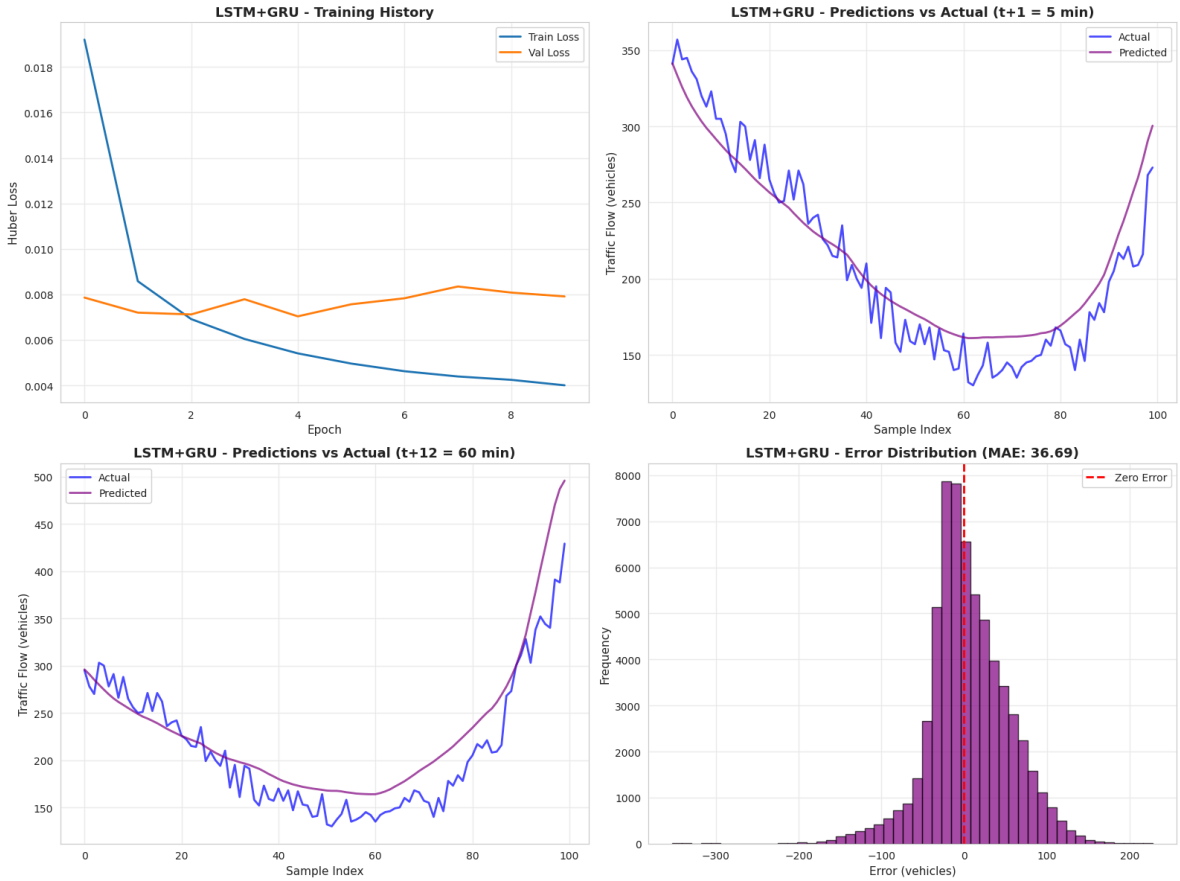
```
================================================================================
EVALUATING LSTM+GRU MODEL
================================================================================

✓ LSTM+GRU Results:
  MAE: 36.69 vehicles
  RMSE: 48.85 vehicles
  R²: 0.9206
  MAPE: 9.89%
  Improvement over baseline: 17.0%
```

Visualize LSTM+GRU Results

```
print("\n📊 Generating LSTM+GRU visualizations...")

fig, axes = plt.subplots(2, 2, figsize=(16, 12))

# Training history
axes[0, 0].plot(history_lstm_gru.history['loss'], label='Train Loss', linewi
axes[0, 0].plot(history_lstm_gru.history['val_loss'], label='Val Loss', line
axes[0, 0].set_xlabel('Epoch', fontsize=11)
axes[0, 0].set_ylabel('Huber Loss', fontsize=11)
axes[0, 0].set_title('LSTM+GRU - Training History', fontsize=13, fontweight=
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

# Predictions vs Actual (t+1)
axes[0, 1].plot(y_test[:sample_size, 0], label='Actual', linewidth=2, color=
axes[0, 1].plot(y_pred_lstm_gru[:sample_size, 0], label='Predicted', linewid
axes[0, 1].set_xlabel('Sample Index', fontsize=11)
axes[0, 1].set_ylabel('Traffic Flow (vehicles)', fontsize=11)
axes[0, 1].set_title('LSTM+GRU - Predictions vs Actual (t+1 = 5 min)', fonts
```

```
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

# Predictions vs Actual (t+12)
axes[1, 0].plot(y_test[:sample_size, -1], label='Actual', linewidth=2, color
axes[1, 0].plot(y_pred_lstm_gru[:sample_size, -1], label='Predicted', linewi
axes[1, 0].set_xlabel('Sample Index', fontsize=11)
axes[1, 0].set_ylabel('Traffic Flow (vehicles)', fontsize=11)
axes[1, 0].set_title('LSTM+GRU - Predictions vs Actual (t+12 = 60 min)', fon
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)

# Error distribution
errors_lstm_gru = (y_test - y_pred_lstm_gru).flatten()
axes[1, 1].hist(errors_lstm_gru, bins=50, color='purple', edgecolor='black',
axes[1, 1].axvline(0, color='red', linestyle='--', linewidth=2, label='Zero
axes[1, 1].set_xlabel('Error (vehicles)', fontsize=11)
axes[1, 1].set_ylabel('Frequency', fontsize=11)
axes[1, 1].set_title(f'LSTM+GRU - Error Distribution (MAE: {mae_lstm_gru:.2f
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('LSTM_GRU_complete_analysis.png', dpi=300, bbox_inches='tight')
print("✓ Saved: LSTM_GRU_complete_analysis.png")
plt.show()
```

```
📊 Generating LSTM+GRU visualizations...
✓ Saved: LSTM_GRU_complete_analysis.png
```



Model Comparison Table

```
print("\n" + "="*80)
print("FINAL COMPARISON - ALL MODELS")
print("="*80)

# Create comparison dataframe
comparison_df = pd.DataFrame({
    'Model': ['Baseline', 'GRU', 'LSTM', 'CNN+GRU', 'LSTM+GRU'],
```

```
    'MAE': [mae_baseline, mae_gru, mae_lstm, mae_cnn_gru, mae_lstm_gru],
    'RMSE': [rmse_baseline, rmse_gru, rmse_lstm, rmse_cnn_gru, rmse_lstm_gru
    'R²': [r2_baseline, r2_gru, r2_lstm, r2_cnn_gru, r2_lstm_gru],
    'MAPE (%)': [mape_baseline, mape_gru, mape_lstm, mape_cnn_gru, mape_lstm
})

# Calculate improvement
comparison_df['Improvement'] = [
    '0%',
    f"{((mae_baseline - mae_gru) / mae_baseline * 100):.1f}%",
    f"{((mae_baseline - mae_lstm) / mae_baseline * 100):.1f}%",
    f"{((mae_baseline - mae_cnn_gru) / mae_baseline * 100):.1f}%",
    f"{((mae_baseline - mae_lstm_gru) / mae_baseline * 100):.1f}%"
]

print("\n" + comparison_df.to_string(index=False))

# Find best model
best_idx = comparison_df['MAE'].idxmin()
best_model_name = comparison_df.loc[best_idx, 'Model']
best_mae = comparison_df.loc[best_idx, 'MAE']

print(f"\n🏆 Best Model: {best_model_name} (MAE: {best_mae:.2f} vehicles)")

# Save results
comparison_df.to_csv('model_comparison_results.csv', index=False)
print("\n✓ Results saved to: model_comparison_results.csv")
```

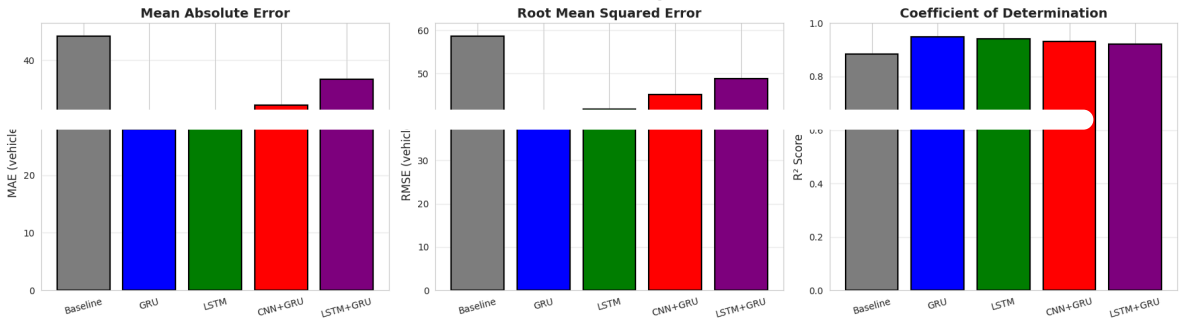
```
================================================================================
FINAL COMPARISON - ALL MODELS
================================================================================

   Model       MAE      RMSE       R²  MAPE (%) Improvement
Baseline 44.210461 58.710169 0.885274 12.325893          0%
     GRU 27.998878 39.330717 0.948513  7.100603       36.7%
    LSTM 30.414213 41.807700 0.941824  8.164198       31.2%
 CNN+GRU 32.154469 45.107680 0.932277  7.947348       27.3%
LSTM+GRU 36.692276 48.853881 0.920561  9.890288       17.0%

🏆 Best Model: GRU (MAE: 28.00 vehicles)

✓ Results saved to: model_comparison_results.csv
```

Comparison Visualization - Overall Performance

```
print("\n📊 Creating comparison visualizations...")

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

models = comparison_df['Model'].tolist()
maes = comparison_df['MAE'].tolist()
```

```
rmses = comparison_df['RMSE'].tolist()
r2s = comparison_df['R²'].tolist()

colors = ['gray', 'blue', 'green', 'red', 'purple']

# MAE
axes[0].bar(models, maes, color=colors, edgecolor='black', linewidth=1.5)
axes[0].set_ylabel('MAE (vehicles)', fontsize=12)
axes[0].set_title('Mean Absolute Error', fontsize=14, fontweight='bold')
axes[0].grid(True, alpha=0.3, axis='y')
axes[0].tick_params(axis='x', rotation=15)

# RMSE
axes[1].bar(models, rmses, color=colors, edgecolor='black', linewidth=1.5)
axes[1].set_ylabel('RMSE (vehicles)', fontsize=12)
axes[1].set_title('Root Mean Squared Error', fontsize=14, fontweight='bold')
axes[1].grid(True, alpha=0.3, axis='y')
axes[1].tick_params(axis='x', rotation=15)

# R²
axes[2].bar(models, r2s, color=colors, edgecolor='black', linewidth=1.5)
axes[2].set_ylabel('R² Score', fontsize=12)
axes[2].set_title('Coefficient of Determination', fontsize=14, fontweight='b
axes[2].set_ylim([0, 1])
axes[2].grid(True, alpha=0.3, axis='y')
axes[2].tick_params(axis='x', rotation=15)

plt.tight_layout()
plt.savefig('COMPARISON_overall_performance.png', dpi=300, bbox_inches='tigh
print("✓ Saved: COMPARISON_overall_performance.png")
plt.show()
```

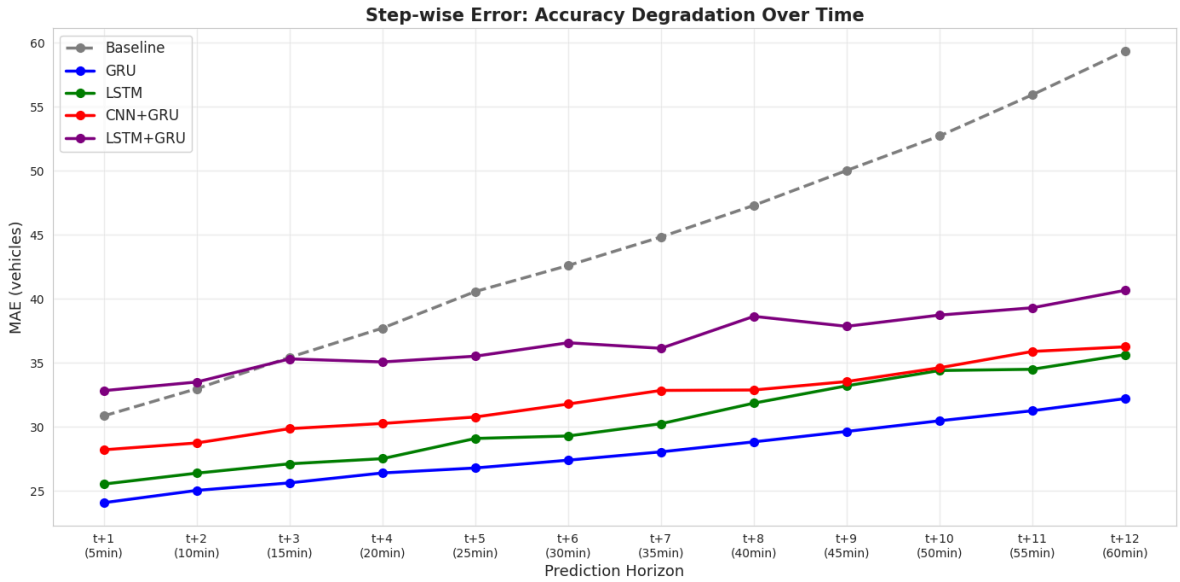
```
📊 Creating comparison visualizations...
✓ Saved: COMPARISON_overall_performance.png
```



Comparison Visualization - Step-wise Error

```
print("\n📊 Creating step-wise error comparison...")

plt.figure(figsize=(14, 7))

time_labels = [f't+{i+1}\n({(i+1)*5}min)' for i in range(OUTPUT_HORIZON)]

# Plot all models
plt.plot(time_labels, step_maes_baseline, marker='o', linewidth=2.5, markers
         label='Baseline', color='gray', linestyle='--')
plt.plot(time_labels, step_maes_gru, marker='o', linewidth=2.5, markersize=7
         label='GRU', color='blue')
plt.plot(time_labels, step_maes_lstm, marker='o', linewidth=2.5, markersize=
         label='LSTM', color='green')
plt.plot(time_labels, step_maes_cnn_gru, marker='o', linewidth=2.5, markersi
         label='CNN+GRU', color='red')
plt.plot(time_labels, step_maes_lstm_gru, marker='o', linewidth=2.5, markers
         label='LSTM+GRU', color='purple')

plt.xlabel('Prediction Horizon', fontsize=13)
plt.ylabel('MAE (vehicles)', fontsize=13)
plt.title('Step-wise Error: Accuracy Degradation Over Time', fontsize=15, fo
plt.legend(fontsize=12, loc='upper left')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('COMPARISON_stepwise_error.png', dpi=300, bbox_inches='tight')
print("✓ Saved: COMPARISON_stepwise_error.png")
plt.show()
```

```
📊 Creating step-wise error comparison...
✓ Saved: COMPARISON_stepwise_error.png
```



Comparison Visualization - Error Distributions

```
print("\n📊 Creating error distribution comparison...")

fig, axes = plt.subplots(2, 3, figsize=(18, 10))
axes = axes.flatten()

# Baseline
errors_baseline = (y_test - y_pred_baseline).flatten()
axes[0].hist(errors_baseline, bins=50, color='gray', edgecolor='black', alph
axes[0].axvline(0, color='red', linestyle='--', linewidth=2)
axes[0].set_xlabel('Error (vehicles)', fontsize=10)
axes[0].set_ylabel('Frequency', fontsize=10)
axes[0].set_title(f'Baseline (MAE: {mae_baseline:.2f})', fontsize=12, fontwe
axes[0].grid(True, alpha=0.3)

# GRU
axes[1].hist(errors_gru, bins=50, color='blue', edgecolor='black', alpha=0.7
axes[1].axvline(0, color='red', linestyle='--', linewidth=2)
axes[1].set_xlabel('Error (vehicles)', fontsize=10)
axes[1].set_ylabel('Frequency', fontsize=10)
axes[1].set_title(f'GRU (MAE: {mae_gru:.2f})', fontsize=12, fontweight='bold
```